جلسه‌ای برای یادگیری – از صفر تا صد

توضیح کامل کد MainActivity

اپلیکیشن رویداد دانشگاهی – پروژه‌محور

MainActivity.java

```java
public class MainActivity extends AppCompatActivity {
    // Android University Events App
}
```

AppCompatActivity | Android Studio | Java

فهرست مطالب

۱۴ موضوع

۱ این فایل اصلاً چیست؟

۲ خط اول: `package`

۳ بخش `import`ها — ابزارهایی که قرض می‌گیریم

۴ کلاس اصلی و وراثت

۵ متغیرهای بالای کلاس

۶ متد `onCreate` — قلب تپنده صفحه

۷ ساخت یک رویداد نمونه

۸ `setOnClickListener` — واکنش به کلیک

۹ متد `showAddEventDialog` — فرم افزودن رویداد

۱۰ متد `refreshEventList` — نمایش لیست روی صفحه

۱۱ متد `openEventDetails` — رفتن به صفحه جزئیات

۱۲ متد `showDeleteConfirmation` — تأیید حذف

۱۳ جمع‌بندی سریع برای سر کلاس

۱۴ چند سؤال احتمالی کلاس

۰۱ | این فایل اصلاً چیست؟

این فایل یک کلاس جاوا به نام `MainActivity` است. اسمش هم واضح است: `Activity` اصلی برنامه.

`Activity` چیست؟

`Activity` یعنی «یک صفحه از برنامه». وقتی برنامه را باز می‌کنی و یک صفحه می‌بینی، آن صفحه یک `Activity` است.

تشبیه

مثل این می‌ماند که یک کتاب داشته باشی و هر `Activity` یک صفحه از آن کتاب باشد. `MainActivity` صفحه اول کتاب است که با باز شدن برنامه نمایش داده می‌شود.

پکیج

کد بالا در پکیج `com.mehrsamweb.universityevents` قرار دارد. پکیج مثل نام خانوادگی برنامه است که نشان می‌دهد این برنامه متعلق به چه کسی / چه شرکتی است.

۰۲ | خط اول: package

MainActivity.java

```
package com.mehrsamweb.universityevents;
```

این خط می‌گوید: این فایل متعلق به پوشه‌ای به نام `com.mehrsamweb.universityevents` است.

تشبیه

مثل این است که روی دفترت بنویسی «کلاس ۱۰۳، دبیرستان شهید بهشتی». یعنی آدرس این فایل کجاست.

com › mehrsamweb › universityevents › MainActivity.java

۰۳ | بخش importها — یعنی ابزارهایی که قرض می‌گیریم

MainActivity.java

```
import android.content.Intent;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
// ... و چند import دیگر
```

جاوا خودش همه چیز را ندارد. بعضی چیزها را باید از جاهای دیگر قرض بگیریم.

`import` یعنی: من می‌خواهم از این ابزار استفاده کنم.

در دو اسلاید بعد: جدول کامل ابزارهای `import` شده و کار هر کدام به زبان ساده.

مهم‌ترین ابزارهای import شده

ابزار (import)	کارش چیست؟ (به زبان ساده)
Intent	پیک‌هایی که بین صفحه‌ها پیام می‌برند؛ با آن از یک صفحه به صفحه دیگر می‌رویم.
AlertDialog	یک پنجره کوچک که روی صفحه باز می‌شود (مثل پنجره «آیا مطمئنی؟»).
AppCompatActivity	کلاس پدری که به Activity ما می‌گوید «تو یک صفحه‌ای».
Bundle	یک جعبه که اطلاعات را وقتی صفحه می‌سازیم نگه می‌دارد.
View	هر چیزی که روی صفحه دیده می‌شود (دکمه، متن، عکس...) یک View است.
LayoutInflater	ابزاری که فایل XML (طراحی صفحه) را به یک شیء واقعی تبدیل می‌کند.

مهم‌ترین ابزارهای `import` شده

ابزار (import)	کارش چیست؟ (به زبان ساده)
`Button, EditText, TextView, Spinner`	دکمه، کادر نوشتن، متن، لیست کشویی.
`Toast`	یک پیام کوچک که پایین صفحه ظاهر می‌شود و خودش می‌رود.
`ArrayList, Arrays, List`	ابزارهای نگهداری فهرست چیزها.

نمونه واقعی از کد: `Button` `Toast` `ArrayList`

نکته برای کلاس

بگو: `import` یعنی من می‌خواهم از این ابزار آماده استفاده کنم؛ نمی‌خواهم از اول چرخ را اختراع کنم.

۰۴ | کلاس اصلی

```
MainActivity.java
public class MainActivity extends AppCompatActivity {
```

این خط را این‌طور بخوان

من یک کلاس به نام `MainActivity` می‌سازم که وارث `AppCompatActivity` است.

وارث یعنی چه؟

یعنی `MainActivity` همه چیزهایی که `AppCompatActivity` دارد را به ارث می‌برد.
مثل بچه‌ای که از پدرش ارث می‌برد.

`AppCompatActivity` به ما می‌گوید: تو یک `Activity` هستی و این امکانات را داری.

AppCompatActivity

کلاس پدر — همه امکانات پایه

↓

MainActivity

کلاس فرزند — به ارث می‌برد

۰۵ | متغیرهای بالای کلاس

الف) کلیدها (EXTRA)

MainActivity.java

```java
public static final String EXTRA_EVENT_TITLE = "event_title";
public static final String EXTRA_EVENT_LOCATION = "event_location";
// ... EXTRA_EVENT_DATE , EXTRA_EVENT_CATEGORY , EXTRA_EVENT_DESCRIPTION
```

این‌ها برچسب هستند. وقتی می‌خواهیم اطلاعات یک رویداد را به صفحه جزئیات بفرستیم، باید روی هر اطلاعات یک برچسب بزنیم تا صفحه مقصد بداند کدام چیست.

تشبیه

فرض کن می‌خواهی یک بسته پستی بفرستی. روی جعبه می‌نویسی «نام گیرنده»، «آدرس»، «کد پستی». این `EXTRA_ ...`ها همان برچسب‌های روی جعبه هستند.

`static` این کلید مشترک بین همه است و به یک شیء خاص وابسته نیست.

`final` قابل تغییر نیست. یک بار تعریف شد، دیگر عوض نمی‌شود.

۰۵ | متغیرهای بالای کلاس

ب) فهرست رویدادها

ج) محفظه نمایش لیست

MainActivity.java

```
private final ArrayList<Event> eventList = new ArrayList<>();
private LinearLayout layoutEventList;
```

eventList – لیست خالی

این یک لیست خالی است که رویدادها را در خودش نگه می‌دارد. الان خالی است، بعداً پر می‌شود.

layoutEventList – قوطی نمایش

یک قوطی است که روی صفحه قرار دارد و رویدادها داخلش چیده می‌شوند. هنوز چیزی به آن وصل نکردیم؛ در onCreate وصلش می‌کنیم.

مهم

این لیست فقط در حافظه (RAM) است. یعنی اگر برنامه کامل بسته شود، همه چیز پاک می‌شود. در جلسه چهارم با SQLite جایگزین می‌شود – این را حتماً سر کلاس بگو.

private یعنی فقط داخل همین کلاس می‌شود به آن دسترسی داشت.

final یعنی خودِ «لیست» قابل تعویض نیست (ولی محتوایش را می‌توان تغییر داد – می‌توان آیتم اضافه/حذف کرد).

۰۶ | متد onCreate – قلب تپنده صفحه

MainActivity.java

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    ...
}
```

onCreate چیست؟

وقتی یک Activity اولین بار ساخته می‌شود، این متد صدا زده می‌شود. مثل «روشن شدن» یک دستگاه.

@Override یعنی چه؟

من دارم کاری را که پدرم (AppCompatActivity) قبلاً تعریف کرده بود، دوباره می‌نویسم و شخصی‌سازی می‌کنم.

super.onCreate(savedInstanceState)

اول کارهای پایه پدرم را انجام بده، بعد برو سراغ کارهای من. اگر این را ننویسیم، برنامه کرش می‌کند.

۰۶ | setContentView و پیدا کردن دکمه‌ها

onCreate ادامه ▶

تشبیه

`setContentView(R.layout.activity_main)` یعنی: این صفحه را با طرحی که در فایل `activity_main.xml` نوشته‌ام بساز و نشان بده. `R.layout.activity_main` مثل آدرس نقشه ساختمان است؛ `setContentView` می‌گوید: این نقشه را بساز و تحویل بده.

MainActivity.java

```
Button btnShowEvent = findViewById(R.id.btnShowEvent);
Button btnAddEvent = findViewById(R.id.btnAddEvent);
Button btnSettings = findViewById(R.id.btnSettings);
layoutEventList = findViewById(R.id.layoutEventList);
```

`findViewById` یعنی: برو در طرحی که ساختی، دنبال عنصری با این شناسه (`id`) بگرد و آن را به من بده.

↳ مثل این است که در یک خانه دنبال اتاقی با پلاک «۱۰۳» بگردی.

`R.id.btnShowEvent` ← دکمه «مشاهده رویداد»

`R.id.btnAddEvent` ← دکمه «افزودن رویداد»

`R.id.btnSettings` ← دکمه «تنظیمات»

`R.id.layoutEventList` ← قوطی رویدادها

۰۷ | ساخت یک رویداد نمونه

MainActivity.java

```
Event sampleEvent = new Event(
    "کارگاه برنامه‌نویسی اندروید",
    "1405/08/25",
    "کارگاه کامپیوتر",
    "آموزشی",
    "آشنایی با ساخت اپلیکیشن‌های اندرویدی با زبان Java"
);
eventList.add(sampleEvent);
refreshEventList();
```

اینجا یک رویداد نمونه می‌سازیم تا وقتی برنامه باز می‌شود، صفحه خالی نباشد و کاربر ببیند چیزی وجود دارد.

ترتیب اطلاعات:

۱ عنوان | ۲ تاریخ | ۳ محل | ۴ دسته‌بندی | ۵ توضیحات

`eventList.add(...)` – آن را به لیست اضافه می‌کنیم.

`refreshEventList()` – می‌گوییم: حالا لیست را روی صفحه نمایش بده.

۰۸ | setOnClickListener — واکنش به کلیک

MainActivity.java

```
btnShowEvent.setOnClickListener(view -> { ... });
```



این خط می‌گوید: وقتی کاربر روی این دکمه کلیک کرد، این کارها را انجام بده.

Σ `view → { ... }` چیست؟

این یک لاندا (`lambda`) است؛ یعنی یک تابع کوچک بدون نام. به زبان ساده: **«ای دکمه! وقتی کلیک شدی، این دستورها را اجرا کن.»**

`view` همان دکمه‌ای است که کلیک شده (معمولاً به آن نیاز نداریم).

در ادامه: کلیک هر سه دکمه صفحه اصلی ←

۰۸ | داخل کلیک دکمه «مشاهده رویداد»

اول چک می‌کنیم: آیا لیست خالی است؟

MainActivity.java

```
if (eventList.isEmpty()) {
    Toast.makeText(MainActivity.this,
        "هنوز رویدادی ثبت نشده است.",
        Toast.LENGTH_SHORT).show();
    return;
}
openEventDetails(eventList.get(0));
```

اگر خالی بود → یک `Toast` (پیام کوچک) نشان می‌دهیم و `return` می‌کنیم (یعنی از متد بیرون می‌رویم).

اگر خالی نبود → `openEventDetails(eventList.get(0))` یعنی: جزئیات اولین رویداد را باز کن. `get(0)` یعنی عنصر شماره صفر، چون شمارش از صفر شروع می‌شود.

نکته مهم برای کلاس

`Toast.makeText(...).show()` یعنی: یک پیام کوتاه بساز و نمایش بده.

۰۸ | دو دکمه دیگر

ادامه

btnAddEvent

افزودن رویداد

MainActivity.java

```
btnAddEvent.setOnClickListener(view -> showAddEventDialog());
```

یعنی: وقتی کلیک شد، متد `showAddEventDialog` را صدا بزن. ساده!

btnSettings

تنظیمات

MainActivity.java

```
btnSettings.setOnClickListener(view -> {
    Toast.makeText(MainActivity.this,
        "بخش تنظیمات را در جلسه چهارم تکمیل می‌کنیم.",
        Toast.LENGTH_SHORT).show();
});
```

فعلاً تنظیمات کار نمی‌کند؛ فقط یک پیام نشان می‌دهد که «بعداً می‌سازیمش».

۰۹ | متد `showAddEventDialog` — فرم افزودن رویداد

این متد بزرگ‌ترین بخش کد است. تکه‌تکه جلو می‌رویم — **۹ قدم**.

۱ ساختن فرم از روی XML — `inflate`

۲ پیدا کردن اجزای فرم

۳ آماده کردن لیست کشویی — `Spinner`

۴ ساخت `Dialog`

۵ نمایش دیالوگ

۶ ترفند دکمه تأیید دستی

۷ خواندن مقادیر فرم

۸ اعتبارسنجی — `Validation`

۹ ثبت رویداد و بستن دیالوگ

۰۹ | قدم ۱ و ۲ – ساختن فرم و پیدا کردن اجزا

۱ قدم ۱ – ساختن فرم از روی XML

MainActivity.java

```
LayoutInflater inflater = getLayoutInflater();
View dialogView = inflater.inflate(R.layout.dialog_add_event, null);
```

`inflate` یعنی: این فایل XML را بخوان و به یک شیء قابل نمایش تبدیل کن.

۲ قدم ۲ – پیدا کردن اجزای فرم

MainActivity.java

```
EditText edtEventTitle = dialogView.findViewById(R.id.edtEventTitle);
// edtEventDate , edtEventLocation , spnEventCategory , edtEventDescription
```

همان `findViewById` قبلی است، ولی این بار داخل فرم دیالوگ می‌گردیم، نه کل صفحه. `dialogView` یعنی: از داخل این فرم بگرد.

تشبیه

فایل XML مثل یک نقشه لِگو است. `inflater` مثل دستی است که لگوها را سر هم می‌کند و یک سازه واقعی می‌سازد.

`null` یعنی: این فرم را داخل هیچ ظرف خاصی نگذار؛ فعلاً فقط بسازش.

۰۹ | قدم ۳ — آماده کردن لیست کشویی (Spinner)

MainActivity.java

```
List<String> categories = Arrays.asList(
    "سایر" ,"مسابقه" ,"ورزشی" ,"فرهنگی" ,"آموزشی");

ArrayAdapter<String> categoryAdapter = new ArrayAdapter<>(this,
    android.R.layout.simple_spinner_item, categories);
categoryAdapter.setDropDownViewResource(
    android.R.layout.simple_spinner_dropdown_item);
spnEventCategory.setAdapter(categoryAdapter);
```

یک لیست متنی از دسته‌بندی‌ها می‌سازیم.

این سه خط یعنی

۱ یک آداپتور بساز. آداپتور = واسطه بین داده‌ها و ظاهر.

۲ ظاهر گزینه‌ها را تنظیم کن.

۳ آداپتور را به `Spinner` وصل کن.

تشبیه

`Spinner` مثل یک قفسه است. آداپتور مثل کارگری است که کتاب‌ها (داده‌ها) را روی قفسه می‌چیند. `setAdapter` می‌گوید: این کارگر را استخدام کن تا کتاب‌ها را بچیند.

۰۹ | قدم ۴ و ۵ — ساخت و نمایش دیالوگ

MainActivity.java

```
AlertDialog dialog = new AlertDialog.Builder(this)
    .setTitle("افزودن رویداد جدید")
    .setView(dialogView)
    .setPositiveButton("ثبت رویداد", null)
    .setNegativeButton("انصراف",
        (dialogInterface, which) -> { })
    .create();

dialog.show();
```

`setTitle(...)`

عنوان پنجره

`setView(dialogView)`

فرمی که ساختیم را داخل دیالوگ بگذار

`setPositiveButton("ثبت رویداد", null)`

دکمه تأیید. چرا `null`؟ چون فعلاً می‌خواهیم بعداً خودمان مدیریتش کنیم (اسلاید بعد)

`setNegativeButton("انصراف",... )`

دکمه انصراف؛ فقط دیالوگ را می‌بندد

`create()`

دیالوگ را بساز (ولی هنوز نشان نده)

▶ `dialog.show()` → حالا دیالوگ روی صفحه ظاهر می‌شود.

۰۹ | قدم ۶ – چرا `null` گذاشتیم و بعد دکمه را دستی تنظیم کردیم؟

MainActivity.java

```
dialog.getButton(AlertDialog.BUTTON_POSITIVE)
    .setOnClickListener(view -> { ... });
```

مشکل

- ✕ اگر از اول `setPositiveButton` را با کد ثبت تنظیم می‌کردیم، `AlertDialog` بعد از هر کلیک خودش دیالوگ را می‌بست – حتی اگر ورودی نامعتبر بود!
- ✕ کاربر مجبور می‌شد دوباره باز کند.

راه‌حل

- ✓ اول `null` می‌گذاریم، بعد از `show()` دکمه را می‌گیریم و خودمان `OnClickListener` را تنظیم می‌کنیم.
- ✓ این‌طور اگر ورودی نامعتبر بود، دیالوگ باز می‌ماند و کاربر می‌تواند اصلاح کند.
- ✓ فقط وقتی همه چیز درست بود، `dialog.dismiss()` را صدا می‌زنیم.

نکته کلیدی برای کلاس

این یک ترفند مهم است. حتماً سر کلاس بگو چرا این کار را کردیم.

۰۹ | قدم ۷ و ۸ – خواندن مقادیر و اعتبارسنجی

1 قدم ۷ – خواندن مقادیر فرم

MainActivity.java

```
String title = edtEventTitle.getText()
    .toString().trim();
```

`getText()` متن داخل کادر را بگیر

↓

`toString()` آن را به String تبدیل کن

↓

`trim()` فاصله‌های اضافی ابتدا و انتها را حذف کن

همین کار برای تاریخ، محل، دسته‌بندی و توضیحات هم انجام می‌شود. برای Spinner: `String category = spnEventCategory.getSelectedItem().toString();` یعنی گزینه‌ای که کاربر انتخاب کرده را بگیر.

2 قدم ۸ – اعتبارسنجی (Validation)

MainActivity.java

```
if (title.isEmpty()) {
    edtEventTitle.setError("لطفاً عنوان رویداد را وارد کنید.");
    edtEventTitle.requestFocus();
    return;
}
```

`isEmpty()`

آیا خالی است؟

`setError(...)`

زیر کادر یک خطای قرمز نشان بده

`requestFocus()`

مکان‌نما (cursor) را ببر داخل همان کادر تا کاربر مستقیم بتواند تایپ کند

`return`

از متد بیرون برو (یعنی ثبت نکن)

همین کار برای تاریخ و محل هم تکرار می‌شود. (توضیحات اجباری نیست.)

۰۹ | قدم ۹ – ثبت رویداد

MainActivity.java

```
Event newEvent = new Event(title, date, location,
    category, description);
eventList.add(newEvent);
refreshEventList();

Toast.makeText(MainActivity.this,
    "رویداد با موفقیت ثبت شد.",
    Toast.LENGTH_SHORT).show();
dialog.dismiss();
```

۱ یک شیء `Event` جدید بساز.

۲ به لیست اضافه کن.

۳ لیست روی صفحه را دوباره بساز (تا رویداد جدید دیده شود).

۴ پیام موفقیت نشان بده.

۵ دیالوگ را ببند.

۱۰ | متد `refreshEventList` — نمایش لیست روی صفحه

MainActivity.java

```
layoutEventList.removeAllViews();
```

اول همه چیزهای قبلی را پاک می‌کنیم. چرا؟ چون اگر پاک نکنیم، هر بار که لیست را دوباره می‌سازیم، موارد تکراری روی هم انبار می‌شوند.

تشبیه

مثل این است که بخواهی روی تخته وایت‌برد چیز جدید بنویسی؛ اول باید تخته را پاک کنی.

اگر لیست خالی بود

MainActivity.java

```
if (eventList.isEmpty()) {
    TextView emptyMessage = new TextView(this);
    emptyMessage.setText("هنوز رویدادی ثبت نشده است.");
    emptyMessage.setTextSize(15);
    emptyMessage.setPadding(12, 24, 12, 24);
    layoutEventList.addView(emptyMessage);
    return;
}
```

یک `TextView` می‌سازیم، متن «خالی است» را می‌گذاریم، اندازه و فاصله‌اش را تنظیم می‌کنیم و به قوطی اضافه می‌کنیم. بعد `return` می‌کنیم تا بقیه کد اجرا نشود.

`new TextView(this)` یعنی: یک متن جدید بساز. `this` یعنی: مال همین Activity.

۱۰ | حلقه روی رویدادها و ساخت ظاهر هر آیتم

ادامه

حلقه روی رویدادها

MainActivity.java

```
for (int i = 0; i < eventList.size(); i++) {
    Event event = eventList.get(i);
    ...
}
```

این یک حلقه است: از `i = 0` شروع می‌کنیم و تا وقتی `i` کوچک‌تر از تعداد رویدادها است ادامه می‌دهیم. در هر دور، `event` به یک رویداد اشاره می‌کند.

`LinearLayout.VERTICAL` → یعنی عمودی بچین (زیر هم).

ساخت ظاهر هر آیتم

MainActivity.java

```
LinearLayout itemLayout = new LinearLayout(this);
itemLayout.setOrientation(LinearLayout.VERTICAL);
itemLayout.setPadding(16, 12, 16, 12);
itemLayout.setBackgroundColor(0xFFFFFFFF);
```

یک قوطی عمودی می‌سازیم (چیزها زیر هم می‌روند)، فاصله داخلی می‌دهیم، رنگ سفید می‌گذاریم.

`LinearLayout.HORIZONTAL` → یعنی افقی بچین (کنار هم).

۱۰ | عنوان رویداد

ادامه ساخت آیتم

MainActivity.java

```java
TextView txtTitle = new TextView(this);
txtTitle.setText(event.getTitle());
txtTitle.setTextSize(18);
txtTitle.setTextColor(0xFF17365D);
txtTitle.setTypeface(null,
    android.graphics.Typeface.BOLD);
```

- **متن** = عنوان رویداد
- **اندازه** = ۱۸
- **رنگ** = 0xFF17365D (یک آبی تیره)
- **BOLD** = پُررنگ (ضخیم)

رنگ را چطور بخوانیم؟

0xFF17365D را چطور بخوانیم؟ 0x یعنی هگزادسیمال. FF یعنی شفافیت کامل (کاملاً مات). بقیه (17365D) کد رنگ است. برای کلاس می‌توانی بگویی: یک کد رنگی است مثل کد رنگ در فتوشاپ.

#17365D

0x	FF	17365D
هگزادسیمال	شفافیت کامل (کاملاً مات)	کد رنگ

۱۰ | اطلاعات رویداد

ادامه

MainActivity.java

```
TextView txtInfo = new TextView(this);
txtInfo.setText("تاریخ: " + event.getDate()
        + "\nمحل: " + event.getLocation()
        + "\nدسته‌بندی: " + event.getCategory());
```

اینجا با + رشته‌ها را به هم می‌چسبانیم. \n یعنی «برو خط بعد» (line break). پس اطلاعات در سه خط نمایش داده می‌شوند.

خروجی روی صفحه

تاریخ: 1405/08/25
محل: کارگاه کامپیوتر
دسته‌بندی: آموزشی

itemLayout.addView(txtTitle); و itemLayout.addView(txtInfo); — عنوان و اطلاعات را داخل قوطی itemLayout می‌گذاریم.

۱۰ | دکمه‌های هر آیتم

ادامه

دکمه مشاهده جزئیات

MainActivity.java

```
Button btnDetails = new Button(this);
btnDetails.setText("مشاهده جزئیات");
btnDetails.setOnClickListener(view -> {
    openEventDetails(event);
});
```

یک دکمه می‌سازیم و می‌گوییم: وقتی کلیک شد، جزئیات همین رویداد باز کن.

نکته مهم

اینجا `event` در حلقه است. پس هر دکمه به رویداد خودش وصل می‌شود. اگر این نکته را نگویی، ممکن است کسی فکر کند همه دکمه‌ها یک رویداد را باز می‌کنند.

دکمه حذف رویداد

MainActivity.java

```
Button btnDelete = new Button(this);
btnDelete.setText("حذف رویداد");
btnDelete.setOnClickListener(view -> {
    showDeleteConfirmation(event);
});
```

وقتی کلیک شد، اول تأیید می‌گیریم. (مستقیم حذف نمی‌کنیم!)

`itemLayout.addView(btnDetails);` و `itemLayout.addView(btnDelete);` — دکمه‌ها هم داخل قوطی می‌روند.

۱۰ | چیدمان نهایی هر آیتم — LayoutParams

پایان بخش ۱۰

MainActivity.java

```
LinearLayout.LayoutParams params =
    new LinearLayout.LayoutParams(
        ViewGroup.LayoutParams.MATCH_PARENT,
        ViewGroup.LayoutParams.WRAP_CONTENT);
params.setMargins(0, 0, 0, 16);
layoutEventList.addView(itemLayout, params);
```

MATCH_PARENT

عرض = کل عرض صفحه. مثل این است که بگویی: فرش را تا آخر اتاق پهن کن.

WRAP_CONTENT

ارتفاع = به اندازه محتوا (نه بیشتر). مثل این است که بگویی: فرش فقط به اندازه‌ای که لازم است باشد.

`setMargins(0, 0, 0, 16)` → فاصله از پایین = ۱۶ (تا آیتم‌ها به هم نچسبند).

۱۱ | متد `openEventDetails` — رفتن به صفحه جزئیات

MainActivity.java

```
Intent intent = new Intent(
    MainActivity.this, EventDetailActivity.class);
```

MainActivity.java

```
intent.putExtra(EXTRA_EVENT_TITLE, event.getTitle());
intent.putExtra(EXTRA_EVENT_LOCATION, event.getLocation());
intent.putExtra(EXTRA_EVENT_DATE, event.getDate());
intent.putExtra(EXTRA_EVENT_CATEGORY, event.getCategory());
intent.putExtra(EXTRA_EVENT_DESCRIPTION,
    event.getDescription());
```

تشبیه

`Intent` مثل یک تاکسی است. می‌گویی: از اینجا (مبدأ) به آنجا (مقصد) برو.

`putExtra` یعنی: این اطلاعات را با برچسبش داخل تاکسی بگذار تا به مقصد برساند.

مثل این است که در تاکسی یک جعبه بگذاری و روی آن بنویسی «عنوان»، «تاریخ» و... . صفحه مقصد با همان برچسب‌ها (`EXTRA_...`) اطلاعات را بیرون می‌آورد.

`startActivity(intent);` → تاکسی را راه بینداز! یعنی صفحه مقصد باز شود.

۱۲ | متد showDeleteConfirmation — تأیید حذف

MainActivity.java

```
new AlertDialog.Builder(this)
    .setTitle("حذف رویداد")
    .setMessage("آیا از حذف رویداد «"
        + event.getTitle() + "» مطمئن هستید؟")
    .setPositiveButton("حذف", (dialog, which) -> {
        eventList.remove(event);
        refreshEventList();
        Toast.makeText(MainActivity.this,
            "رویداد حذف شد.",
            Toast.LENGTH_SHORT).show();
    })
    .setNegativeButton("انصراف",
        (dialog, which) -> { dialog.dismiss(); })
    .show();
```

این یک دیالوگ ساده است

۱ **عنوان:** «حذف رویداد»

۲ **پیام:** «آیا از حذف رویداد «...» مطمئن هستید؟» (اسم رویداد را وسط پیام می‌گذاریم)

۳ **دکمه «حذف»:** رویداد را از لیست پاک کن، لیست را بازسازی کن، پیام موفقیت بده

۴ **دکمه «انصراف»:** فقط دیالوگ را ببند (هیچ چیز حذف نمی‌شود)

۵ `.show()` → دیالوگ را نمایش بده

چرا تأیید می‌گیریم؟

چون حذف برگشت‌ناپذیر است. اگر کاربر اشتباهی کلیک کند، داده از دست می‌رود. این یک اصل مهم UX است.

۱۳ | جمع‌بندی سریع

اگر بخواهی در ۲ دقیقه کل کد را موجه شوی:

۱ این فایل صفحه اصلی برنامه است (`MainActivity`).

۲ با `onCreate` صفحه ساخته می‌شود و دکمه‌ها پیدا می‌شوند.

۳ یک رویداد نمونه می‌سازیم تا صفحه خالی نباشد.

۴ با کلیک روی «افزودن»، یک دیالوگ فرم باز می‌شود؛ اطلاعات را می‌گیریم، اعتبارسنجی می‌کنیم و اگر درست بود به لیست اضافه می‌کنیم.

۵ با `refreshEventList` لیست را روی صفحه می‌سازیم (هر رویداد = یک قوطی عمودی با عنوان، اطلاعات، دکمه جزئیات و دکمه حذف).

۶ با `Intent` اطلاعات را به صفحه جزئیات می‌فرستیم (با `putExtra`).

۷ برای حذف، اول تأیید می‌گیریم، بعد حذف می‌کنیم.

۸ همه چیز فعلاً در حافظه است؛ با بستن برنامه پاک می‌شود. در جلسه بعد با `SQLite` جایگزین می‌شود.

۱۴ | چند سؤال احتمالی

س: چرا `eventList` را `final` گذاشتیم؟

ج: چون نمی‌خواهیم خودِ «لیست» عوض شود، فقط محتوایش تغییر کند (اضافه/حذف). `final` جلوی تعویض شیء را می‌گیرد، نه تغییر محتوا.

س: `R.id.btnAddEvent` چیست؟

ج: `R` یک کلاس تولیدشده خودکار است که همه شناسه‌ها و منابع را در خود دارد. `R.id` یعنی «شناسه‌ها»، `R.layout` یعنی «طرح‌ها».

س: چرا `AlertDialog` را با `null` ساختیم بعد دکمه را دستی وصل کردیم؟

ج: تا اگر ورودی نامعتبر بود، دیالوگ بسته نشود و کاربر بتواند اصلاح کند.

۱۴ | چند سؤال احتمالی

س: `trim()` چه کار می‌کند؟

ج: فاصله‌های اضافی ابتدا و انتها را حذف می‌کند. مثلاً `"  سلام  "` می‌شود `"سلام"`. برای اعتبارسنجی مهم است (وگرنه یک فاصله هم «متن» حساب می‌شود).

س: `MATCH_PARENT` و `WRAP_CONTENT` چه فرقی دارند؟

ج: `MATCH_PARENT` = کل فضای موجود را پر کن. `WRAP_CONTENT` = فقط به اندازه محتوا.

س: چرا در حلقه `event` را داخل کلیک استفاده کردیم؟

ج: چون هر دکمه باید به رویداد خودش وصل شود، نه به یک رویداد مشترک.

پایان جلسه

موفق باشید!

حالا می‌توانی کد MainActivity را قدم‌به‌قدم یادبگیری.

۸ نکته جمع‌بندی را مرور کن

ترفند null دیالوگ را فراموش نکن

جلسه چهارم: SQLite

MainActivity.java

```
// see you in session 4
Intent next = new Intent(this, SQLiteSession.class);
```